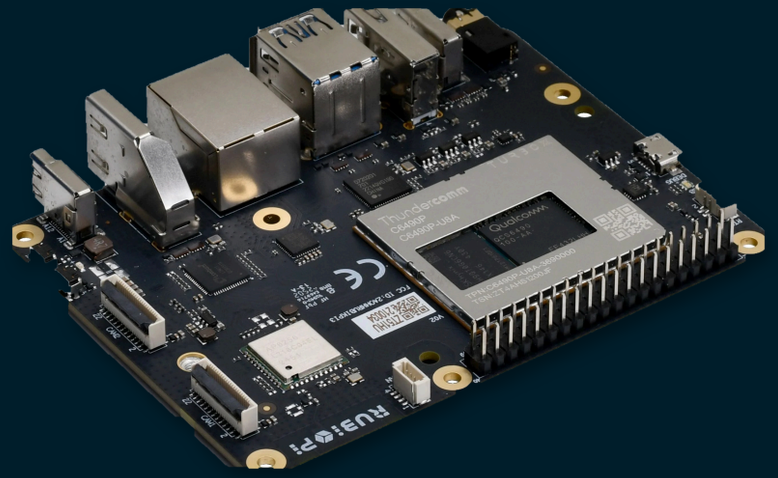
# What Should a Local Agent Remember?

Saving state for agents running at the edge



**Eoin Jordan**

Senior Developer Relations Engineer, Edge Impulse, part of Qualcomm



40 minutes · live on a RUBIK Pi 3 (Qualcomm QCS6490) · Ubuntu 24.04 · MongoDB 9.0 Community

# Who's talking



## Eoin Jordan

Senior Developer Relations Engineer, Edge Impulse, part of Qualcomm · five years

Eoin works in Developer Relations at Edge Impulse, with over 13 years of experience in networking, cloud, edge and IoT technologies. He is passionate about edge intelligence, is pursuing a PhD on the subject at the Insight Centre, University of Galway, and educates the community on this emerging field.

Beyond his professional and academic career, Eoin is engaged in outreach programmes in his local community, contributing to projects that advocate for the democratisation of technology and foster an inclusive and diverse environment.

Before Edge Impulse: eight years at IBM Galway, five as technical lead on distributed and cloud-native systems. Built: the MCP server this talk starts from, and MongoDBSimCity, an explorable model of a mongod.

MongoDB skill badges, October 2026: Building AI Agents with MongoDB · AI and Innovation: How MongoDB Enables a Resilient AI Strategy · Schema Design Patterns (in progress)

github.com/eoinjordan · linkedin.com/in/eoin-jordan · eoinjordan.github.io/MongoDBSimCity

# After this session you will be able to

1

## Model and index the state of a workflow

as documents the agent can read in one go, with the index each question needs

2

## Decide what the agent remembers

what reacts to change, what expires, and what it must be able to explain

*For engineers who have built local edge AI systems and are tired of fragile ones: state in files and prompts, restarts that forget, actions nobody can explain.*

**Full disclosure.** Known for years: agents and tools, MCP servers, edge ML on Qualcomm silicon, distributed systems. New in the last three weeks: mongosh, PyMongo, time series and TTL, change streams in anger, the LangGraph checkpointer. The two MongoDB skill badges are from this month.

**Why this talk:** a teaching exercise built on one real problem, to show MongoDB doing the job of an agent's memory with features you already run. One board, not a fleet; where that matters I say so. Built on MongoDB's own assets and resources wherever they exist; see the references.

**Follow the database side live**

MongoDBSimCity: walk through a MongoDB 9.0 server in your browser while we talk

eoinjordan.github.io/MongoDBSimCity

# Agenda

1. **Three questions for you** — 2 min
   a poll, so the talk starts from your answers
2. **Where it broke** — 6 min
   the demo that started this, and what leaked when it had no memory
3. **Learning objective 1: model and index the state** — 12 min
   the questions, one document, embed or reference, the index each question needs
4. **Learning objective 2: decide what the agent remembers** — 13 min
   what reacts to change, what expires, what must be explainable
5. **The whole picture, and what to take back** — 4 min
   the architecture, one action per role, three things to remember

*Three live demos on the RUBIK Pi along the way, two pauses for you, then fifteen minutes of questions with the Pi still on.*

# Before we start: three questions for you

1

## Are you running an agent in production?

| Yes | Piloting | Not yet |
|---|---|---|

2

## Where does your agent's state live today?

| In the prompt | Files or cache | A database | Nowhere, honestly |
|---|---|---|---|

3

## What has hurt most?

| Lost IDs / orphaned jobs | Polling and latency | Token cost | Nobody can explain what it did |
|---|---|---|---|

### Zoom poll is open

Or type 1 / 2 / 3 with your answer in the chat, e.g. “2: prompt”.

`2: prompt`

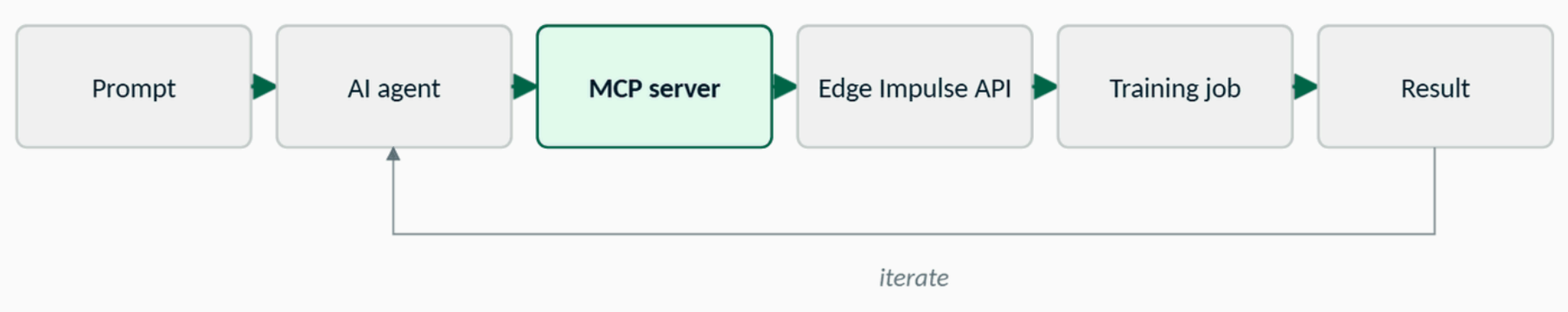
`3: polling`

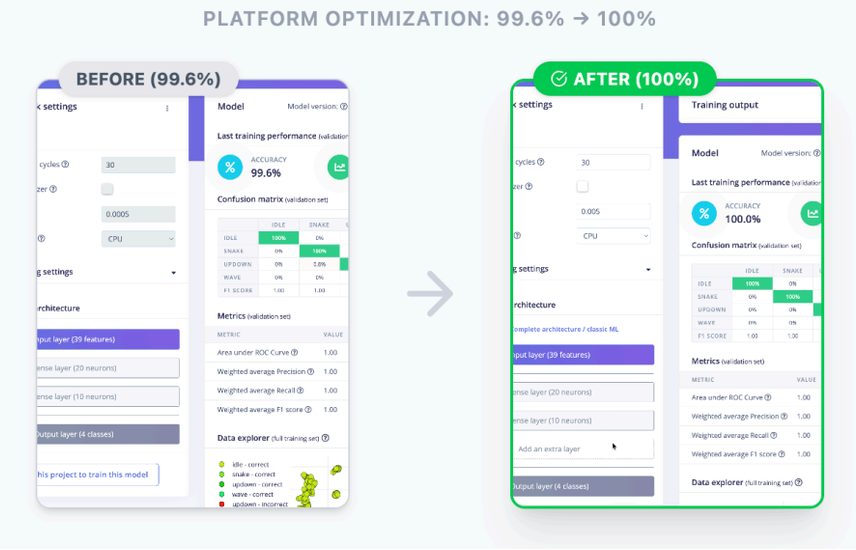
*We come back to your answers twice during the talk.*

# What I actually built

The write-up

A natural-language control plane over Edge Impulse Studio




*"Train this project, optimise for accuracy under constraints, and tell me what changed."*

What came back: the same project, retrained by the agent under those constraints, 99.6% to 100%, with a summary of what it changed. (Screenshot from the blog post.)

edgeimpulse.com/blog/lessons-learned-from-building-an-agentic-ai-mcp-server-for-edge-impulse

# How I stopped it bluffing

Three moves, before any database was involved:

1

## Stub the whole API

mock first, real jobs later

2

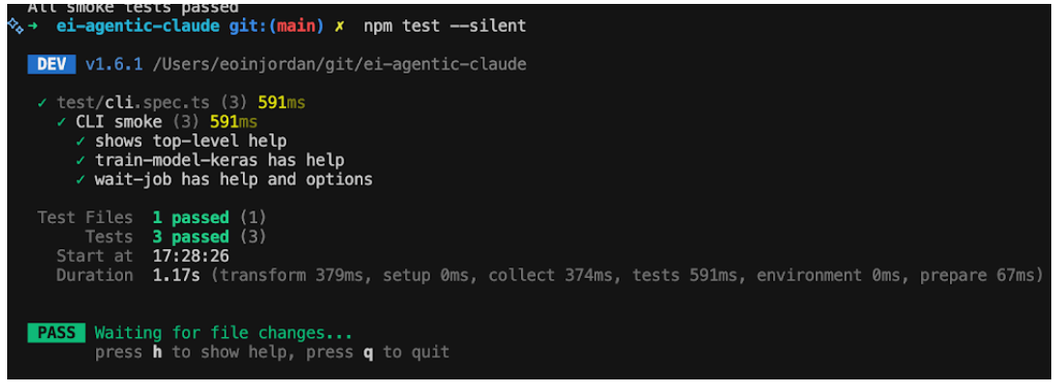
## Test every tool

happy · failure · missing params · edge cases

3

## Add agent-friendly helpers

wait / status / retry · scoping · validation




*The per-tool test run from the blog post: happy path, failure path, missing params, edge cases.*

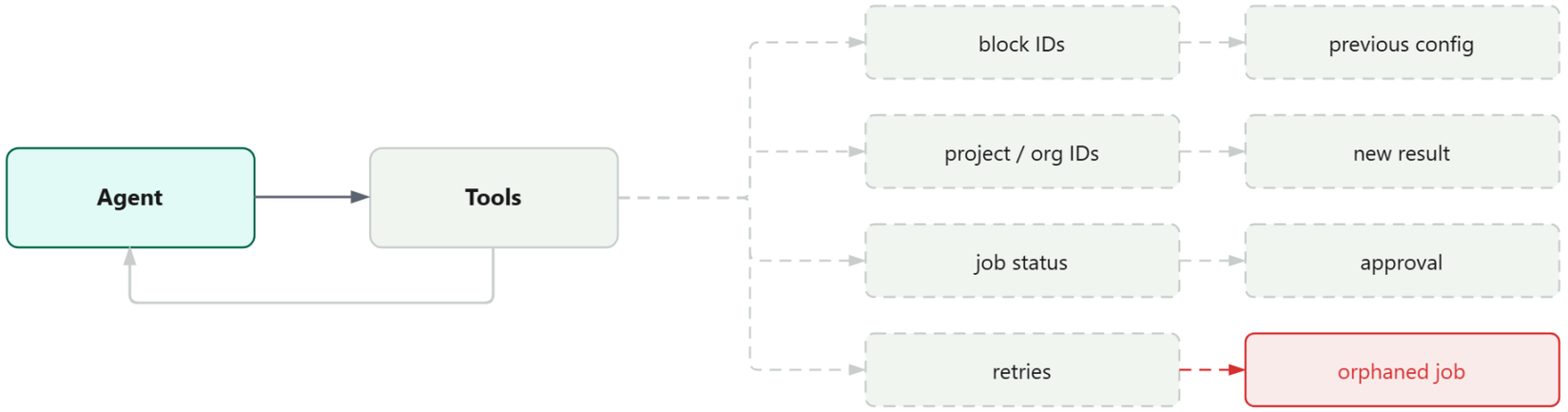
**The key wasn't MCP. It was discipline so the agent couldn't bluff its way through.**

*Remember helper 3: it's the second half of this talk.*

edgeimpulse.com/blog/lessons-learned-from-building-an-agentic-ai-mcp-server-for-edge-impulse

# Then it started leaking state

**IDs everywhere**
**Job timing**
**State management**




IDs everywhere · job timing · state management

**None of this is an LLM problem.**
**It is a state problem.**

# Where does this live?

In the prompt?

In a file?

In a cache?

**In a database?**

# Today we fix that: two objectives

You already run models on devices. New today: short-term and long-term memory as data, and a database that reacts, expires and explains.

1

## Model and index the state

of an agent workflow as documents the agent can read in one go, with the index each question needs

2

## Decide what the agent remembers

short-term versus long-term: what reacts to change, what expires, what must be explainable later

Three parts: the document and its indexes · reacting to change · what to keep, what to expire, what to explain

*Each idea is applied to the same running example, a training run.*

# Start with the questions

What will the system ask when things go wrong?

What runs are active for this project?

What changed since the last good run?

Which jobs are still pending?

Which actions were approved by a human?

What previous failure looks like this one?

**Decide the questions first. The document's shape follows them.**

Relationships, read patterns and write patterns decide what goes where.

# Our first document

live on the RUBIK Pi

Data modelling docs

```
{
  runId:      "run-1842",
  projectId:  "project-512482",
  orgId:      "org-71",
  objective:  { metric: "accuracy", target: 0.92 },
  status:     "training",
  configuration: { windowMs: 2000,
                   learningRate: 0.001 },
  jobs: [
    { jobId: "job-203", type: "training",
      status: "running", startedAt: ... }
  ],
  results:    { accuracy: 0.89, latencyMs: 14 },
  approvals: [
    { action: "train", by: "human", at: ... }
  ],
  createdAt:  ..., updatedAt: ...
}
```

## What's wrong with it?

What grows without bound?

What changes every few seconds?

What needs a history, not a field?

Which query must be fast?

What should not be in here at all?

A useful first model to challenge.

# If the Pi is offline: the first document and its indexes, live

```
{
  runId: 'run-1842',
  projectId: 'project-512482',
  orgId: 'org-71',
  objective: {
    metric: 'accuracy',
    target: 0.92
  },
  status: 'training',
  configuration: {
    windowMs: 2000,
    learningRate: 0.001
  },
  jobs: [
    {
      jobId: 'job-203',
      type: 'training',
      status: 'running',
      startedAt: ISODate('2026-10-16T14:45:46.900Z')
    }
  ],
  results: {
    accuracy: 0.89,
    latencyMs: 14
  },
  approvals: [
    {
      action: 'train',
      by: 'human',
      at: ISODate('2026-10-16T14:44:46.900Z')
    }
  ],
  createdAt: ISODate('2026-10-16T14:44:46.900Z'),
  updatedAt: ISODate('2026-10-16T14:48:46.900Z')
}
ubuntu@ubuntu:~/agentdemo$
```

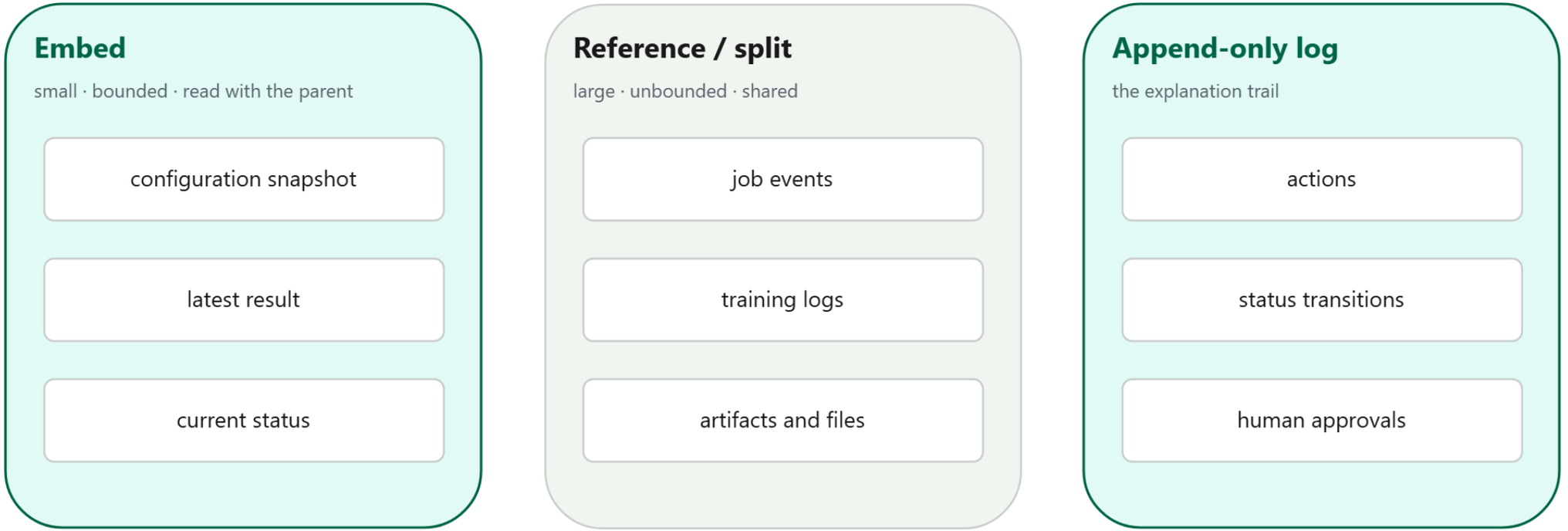
```
Q1  Is anything still training for this project?
    runs.find({ projectId: "project-512482", status: "training" })
    index used    : projectId_1_status_1
    keys examined : 1   docs examined: 1   returned: 1
    → {"runId":"run-1842","status":"training"}

Q2  What happened most recently?
    runs.find({ projectId: "project-512482" }).sort({ updatedAt: -1 }).limit(1)
    index used    : projectId_1_updatedAt_-1
    keys examined : 1   docs examined: 1   returned: 1
    → {"runId":"run-1842","status":"training","updatedAt":"2026-10-16T14:48:46.900Z"}

The trade: every insert and every status update now maintains both indexes.
  indexes on runs: _id_, runId_1, projectId_1_status_1, projectId_1_updatedAt_-1
ubuntu@ubuntu:~/agentdemo$
```

Left: db.runs.findOne() for run-1842 on the RUBIK Pi. Right: the two slide-13 questions with explain(): each hits its index, one key examined.

# Embed, reference, or log?




*Decide by change, growth and access. The reasoning matters more than any rule of thumb.*

# The question creates the index

What the agent asks

```
runs.find({ projectId: "project-512482",
            status: "training" })
```

The index that answers it

```
{ projectId: 1, status: 1 }
```

```
runs.find({ projectId: "project-512482" })
    .sort({ updatedAt: -1 }).limit(1)
```

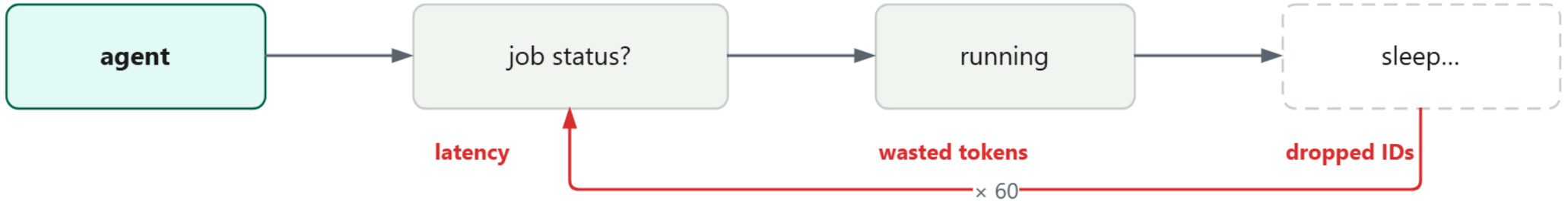
```
{ projectId: 1, updatedAt: -1 }
```

| Reads | Writes | Storage |
| --- | --- | --- |
| **faster** | **slower** | **larger** |

*Equality, then sort, then range. An index is a prediction about how the system will be used. Build them for the questions you actually ask.*

# Then the wait loop hurt

The “train + wait + summarise” skill, as actually implemented




every check is a read · every wait is latency and tokens · every restart is a place to lose the job ID

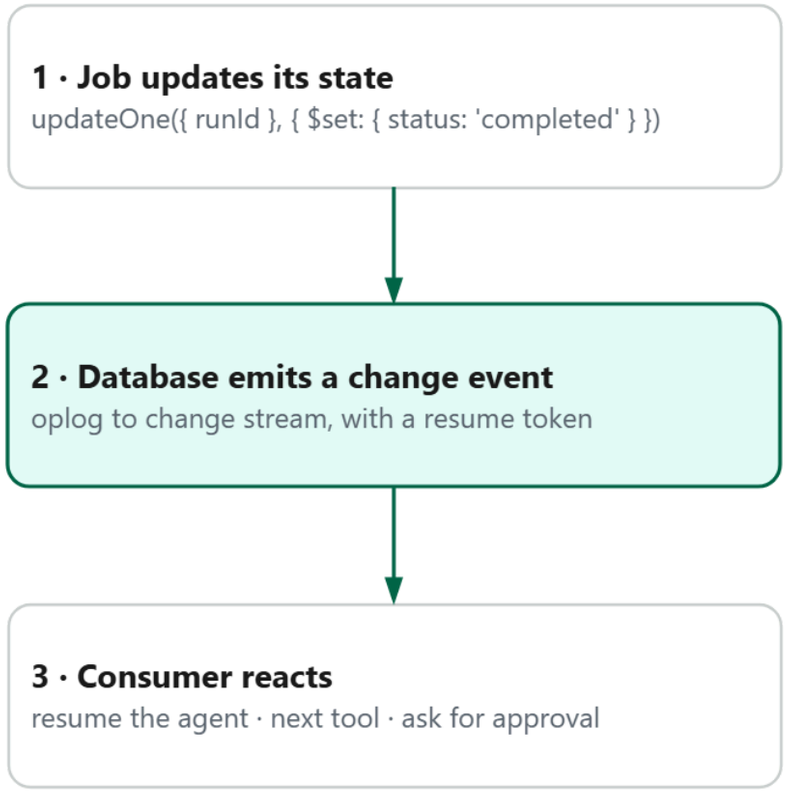
Every check is a read. Every wait is latency and tokens. Every restart is a place to lose the job ID.

**The agent was doing the database's job..**

# From polling to events

live on the RUBIK Pi

Change streams docs



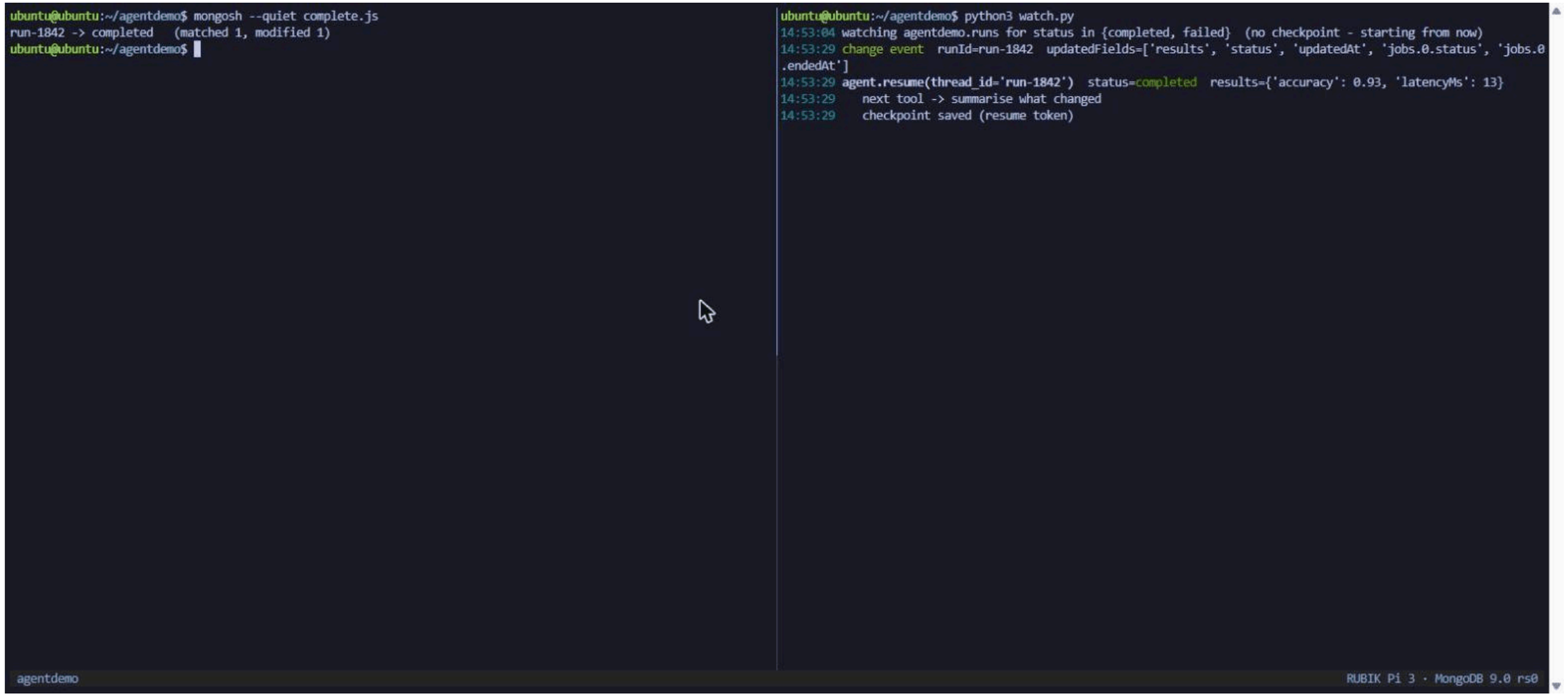

```
pipeline = [{"$match": {
    "operationType": "update",
    "updateDescription.updatedFields.status":
        {"$in": ["completed", "failed"]}
}}]

with runs.watch(pipeline,
                full_document="updateLookup") as cs:
    for change in cs:
        run = change["fullDocument"]
        agent.resume(thread_id=run["runId"])
```

*Still design the failure path: missed events, retries, idempotency, resume.*

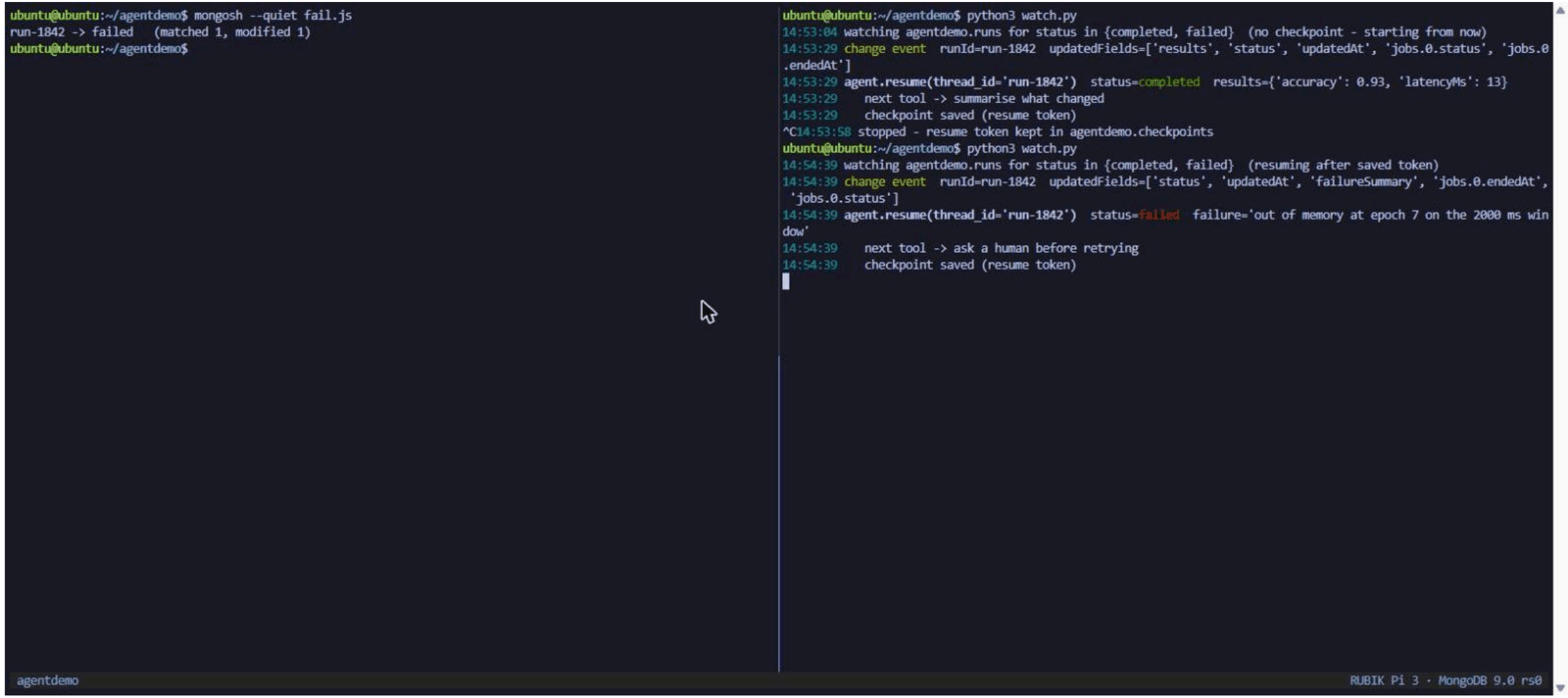
*No polling loop. No lost job IDs. The event carries the document the agent needs next.*

# If the Pi is offline: from polling to events, live




complete.js updates the run; the change stream fires; the consumer resumes the agent by thread_id and saves its resume token.

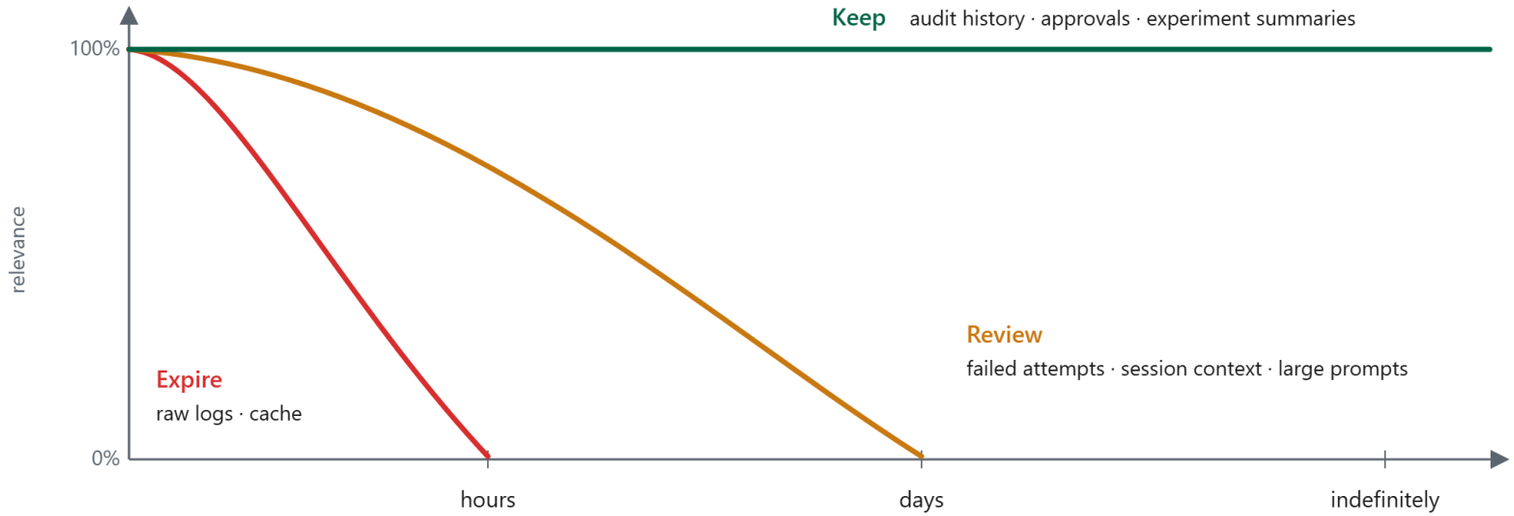
# If the Pi is offline: the consumer survives a restart




The consumer is killed, the run fails while it is down, and the restarted consumer receives the missed event from the saved resume token.

# Memory has a half-life

TTL index docs




```
db.sessions.createIndex(
  { expiresAt: 1 }, { expireAfterSeconds: 0 })
// the writer sets expiresAt per document
```

*Lifetime becomes a schema decision, set when the document is written, not a cron job.*

Short-term memory expires; long-term memory is kept and audited. TTL enforces the first. It never decides which is which.

# If the Pi is offline: memory has a half-life, live

```
ubuntu@ubuntu:~/agentdemo$ mongosh --quiet ttl.js
TTL index on sessions: {"key":{"expiresAt":1},"expireAfterSeconds":0}
   {"sessionId":"sess-1842","expiresAt":"2026-10-16T14:56:12.320Z"}
   {"sessionId":"sess-1841","expiresAt":"2026-10-17T14:55:12.320Z"}
sessions now: 2 — the TTL monitor runs about once a minute; run ttl-check.js in ~90 s
audit is NOT in the expiry list: no TTL index on audit
ubuntu@ubuntu:~/agentdemo$
```

```
sessions now: 1
   {"sessionId":"sess-1841","expiresAt":"2026-10-17T14:56:24.240Z"}
ubuntu@ubuntu:~/agentdemo$
```

```
runs.find({ projectId: "project-512482", "jobs.status": "failed", updatedAt: { $gte: startOfToday } })
{
  runId: 'run-1842',
  status: 'failed',
  failureSummary: 'out of memory at epoch 7 on the 2000 ms window'
}
{
  runId: 'run-1840',
  status: 'failed',
  failureSummary: 'loss diverged after 3 epochs; learning rate too high for 500 ms windows',
  failureEmbedding: [
    0.12,
    -0.44,
    0.31,
    0.08,
    -0.27,
    0.55,
    -0.1,
    0.19
  ]
}

Same collection. Same document. The embedding sits beside the structured fields and is filtered by them.
ubuntu@ubuntu:~/agentdemo$
```

Two sessions written with per-document expiresAt; a minute later the short one is gone and audit has no TTL. Right: the slide-21 structured query, embedding stored beside the fields.

# What would you delete?

Everything the agent wrote this week, in one list. Which of these has to live?

Audit history · Experiment summaries · Approvals
Session context · Raw logs · Failed attempts · Cache

# Not every question is semantic

Vector Search docs

*Failed jobs for project X, today.*

```
runs.find({
projectId: "project-512482",
"jobs.status": "failed",
updatedAt: { $gte: today } })
```

*Previous failures like this one.*

```
runs.aggregate([{ $vectorSearch: {
path: "failureEmbedding",
queryVector: embed(thisFailure),
filter: { projectId: ... } }}])
```

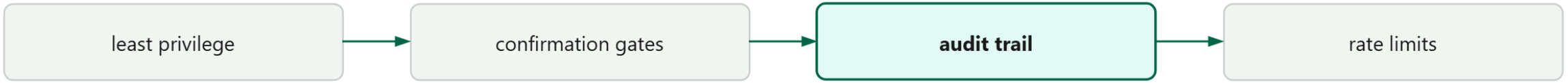



**The same collection and the same document answer a different kind of question.**

A reliable agent needs both: exact operational state, and similarity over messy context, governed by the same rules.

# An agent that can act must be explainable

IDC: the control point



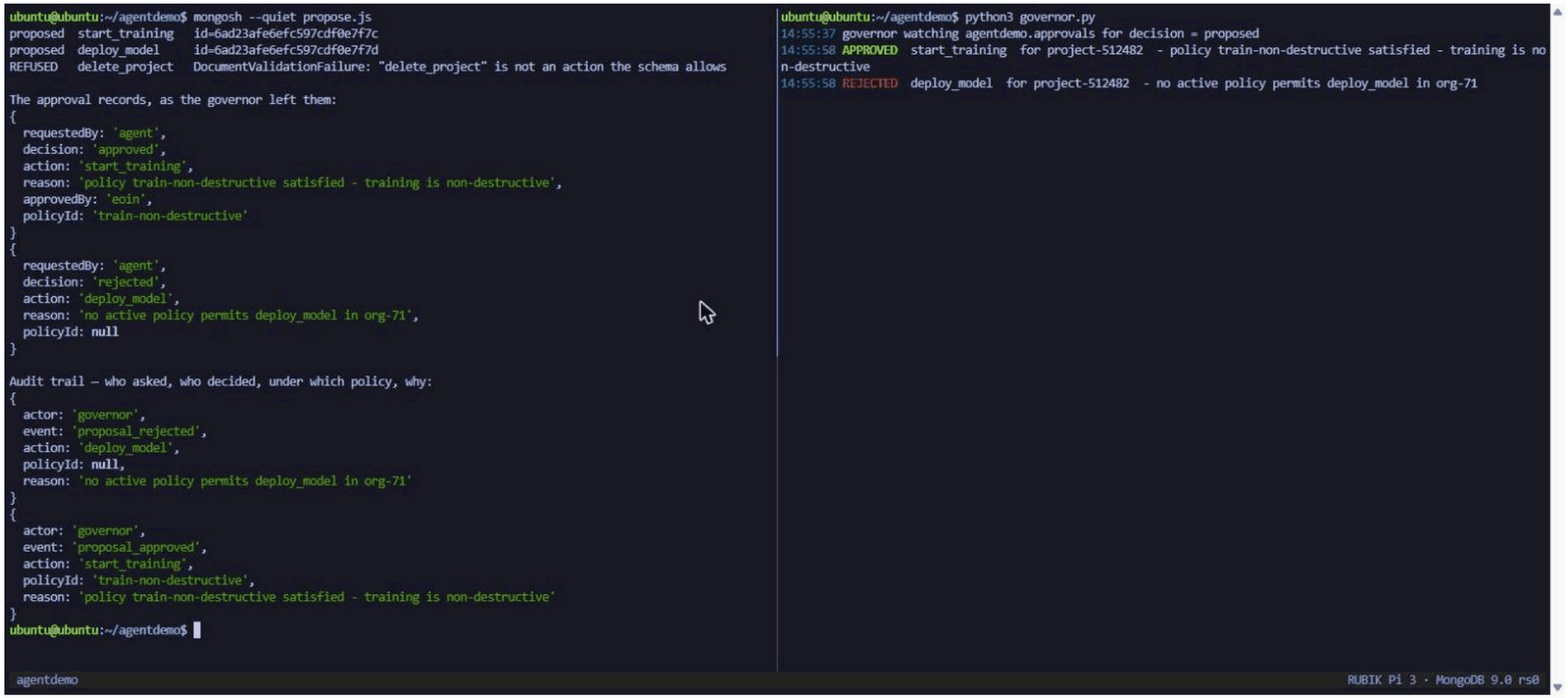

```
{
  action:      "start_training",
  requestedBy: "agent",
  approvedBy:  "eoin",
  scope:       { orgId: "org-71",
                 projectId: "project-512482" },
  decision:    "approved",
  reason:      "non-destructive training run",
  at:          ISODate("...")
}
```

**If a system can act, it must be able to explain how that action was allowed.**

Approvals and audit live in collections with a schema. The database checks the policy before the action fires, so a bad model output cannot become a bad action.

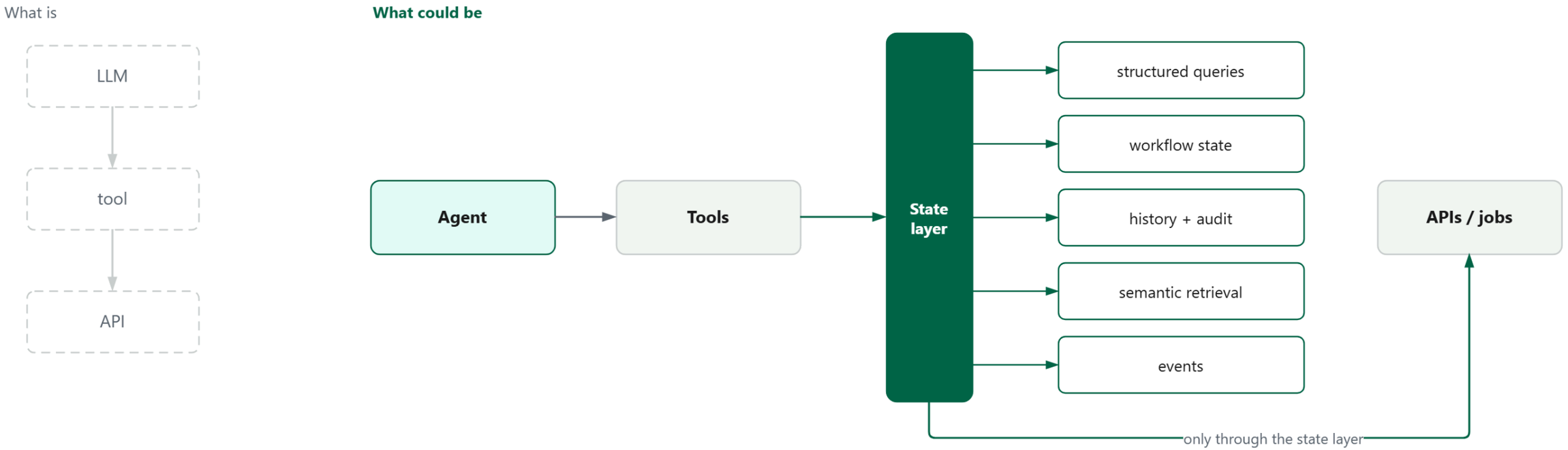
*Guardrails are data.*

# If the Pi is offline: guardrails are data, live




The agent proposes three actions. start_training is approved by policy, deploy_model is rejected because no policy permits it, delete_project is refused by the schema before any decision. Every decision is in audit.

# The resolved architecture




*Giving an agent tools is easy. Giving it reliable state is the hard part.*

# What to take back

| | |
|---|---|
| **Developer** | Give every workflow a document and a thread ID. |
| **Architect** | Put a state layer between tools and the world. |
| **Data / platform lead** | Index from the query log; make lifetimes a schema decision with TTL. |
| **Product / engineering manager** | Ask one question of any agent project: can it explain what it did? |

*Back to your chat answers: which of these is yours?*

# Three takeaways

1

## Model for the workload.

One ticket per table: everything the pass needs about that table in one glance.

2

## Index for the questions.

Hang the tickets by table, then course, so “table twelve, mains” is one reach, not a search.

3

## Treat AI memory as data.

Date the prep, keep the order book, send every plate across the pass.

*Which of the three would have saved you a restart last quarter?*

# Where to go next

Routed by what you need next, one QR each:

## Learn

*Badges and books*

- Skill Badges: Building AI Agents with MongoDB, Schema Design Patterns
- Book: From Systems of Record to Agentic Systems of Action (Packt, 2026)
- The Official MongoDB Guide

learn.mongodb.com/skills · mongodb.com/books

## Ask

*Answered directly*

- Community Forums: engineers answer
- Docs, and the docs assistant
- The four doc pages behind this talk (corner QRs)

mongodb.com/community/forums

## Build

*Give your coding agent the skills*

- MongoDB Agent Skills: schema design, query optimizer, MCP setup
- npx skills add mongodb/agent-skills
- Bundled in the Claude, Codex, Cursor and Gemini plugins

mongodb.com/docs/agent-skills

## Scale

*Customers and partners*

- Your account team
- Or connect with an expert
- Bring the run document: it's the conversation starter

mongodb.com/company/contact

# Thank you

Questions. I can screen-share the RUBIK Pi demo for any of this.

**Code**

github.com/eoinjordan
linkedin.com/in/eoin-jordan

**The model**

eoinjordan.github.io/MongoDBSim

**The write-up**

Lessons learned building an agentic MCP server (Edge Impulse blog)

# A 10-minute lab from this talk

1. Given a messy agent workflow log, identify the state that must persist.
2. Create a first run document and argue what to embed, reference or log.
3. Write two query patterns and derive one compound index, then name its write cost.
4. Replace one polling loop with a change-stream consumer that survives a restart.
5. Add TTL for temporary session data and keep the audit history.

*Build the design habit on a problem the learner has already felt.*

# Agent memory is data: short-term and long-term

LangGraph integration

## Short-term

*State between steps*

Checkpoint per node transition, keyed by the run. Survives a restart. Pauses for human approval.

```
MongoDBSaver(client)
thread_id = run["runId"]
interrupt_before=["tools"]
```

## Long-term

*What the agent learned across runs*

Memories namespaced by project. Looked up by key, or semantically with a vector index.

```
MongoDBStore(...)
namespace = ("project-512482",
             "experiments")
```

*The helpers I bolted on in five evenings are what these libraries now do for you, on top of the same collections.*

# When documents aren't enough

## Documents

*Records and workflow state*

"What is the status of run 1842?"

the operational truth: fast, indexed, audited

## Vectors

*Similarity over messy context*

"Which failures look like this one?"

a query mode beside the record, filtered by it

## Graph / RDF

*Relationships and inference*

"Which devices are affected by this dataset change?"

$graphLookup for bounded hops; a triple store when you need reasoning

*Pick the store by the shape of the question, not the shape of the data.*

# The helpers I wrote by hand now ship

| What I built in five evenings | What MongoDB ships as Agent Skills | |
|---|---|---|
| **Stub the whole API** | **MongoDB MCP Server** | the agent talks to the real database: real schemas, real indexes, no guessing |
| **Test every tool** | **Query Optimizer skill** | checks the index serves the query; finds the equality-sort-range order |
| **Agent-friendly helpers** | **Schema Design + NL Querying skills** | embed vs reference, validation rules, natural language to aggregation |

```
npx skills add mongodb/agent-skills   ·   /schema-design
/query-optimizer
```

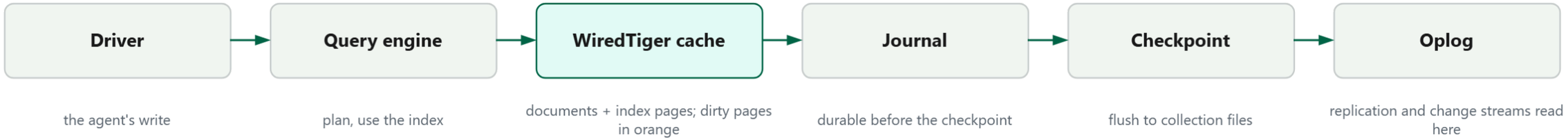
*Skills are the curriculum, written for the agent. Bundled with the MCP Server in the Claude, Codex, Cursor and Gemini plugins.*

# If runs outgrow one node: sharding

| | |
|---|---|
| `runId` | Unique, spreads evenly, but every project query becomes scatter-gather. Wrong key for our questions. |
| `projectId` | Targets both hot queries. Risk: one very busy project is a hot shard. |
| `orgId` | Even coarser. Fine for isolation, worse for balance. |
| `{ projectId, createdAt }` | Targeted and time-ordered within a project; the usual compromise for append-heavy workflows. |
| `hashed projectId` | Best spread, loses range-by-time. Choose when balance matters more than recency scans. |

*Shard by what you query by. Change streams still work across shards; ordering is per-shard.*

# What the server is doing underneath




## Why it matters for the agent

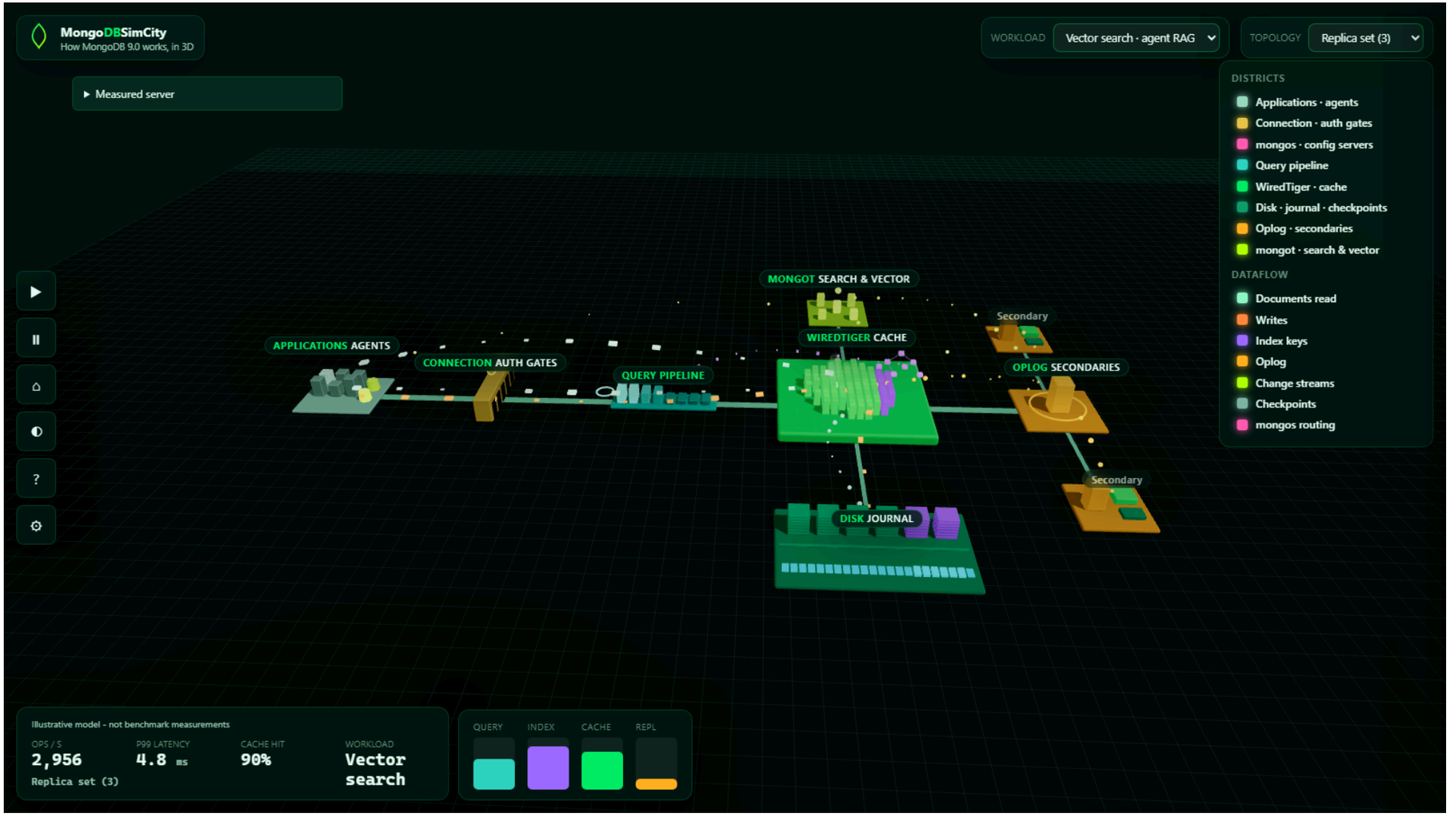
A status write every few seconds dirties a page every few seconds. Unbounded arrays make that page large and the eviction (80% target, 95% trigger) more expensive.

Every index is pages in the same cache as the documents, so the write cost on the index slide is cache pressure, not just disk.

Change streams read the oplog, so the consumer is never on the write path.

*Live view: eoinjordan.github.io/MongoDBSimCity: illustrative numbers, documented thresholds.*

# Backup: the database side, live (MongoDBSimCity)



# Sources

| | |
|---|---|
| **Lessons learned building an agentic MCP server** | edgeimpulse.com/blog/lessons-learned-from-building-an-agentic-ai-mcp-server-for-edge-impulse/ |
| **MongoDB data modelling** | mongodb.com/docs/manual/data-modeling/ |
| **Change streams** | mongodb.com/docs/manual/changeStreams/ |
| **TTL indexes** | mongodb.com/docs/manual/core/index-ttl/ |
| **Atlas Vector Search overview** | mongodb.com/docs/atlas/atlas-vector-search/vector-search-overview/ |
| **LangGraph + MongoDB (checkpointer, store)** | mongodb.com/docs/atlas/ai-integrations/langgraph/ |
| **IDC: Your database just became your AI control point** | mongodb.com/company/blog/news/your-database-just-became-your-ai-control-point |
| **MongoDB Agent Skills** | mongodb.com/docs/agent-skills/ · github.com/mongodb/agent-skills |
| **From Systems of Record to Agentic Systems of Action (Packt, 2026)** | mongodb.com/books |
| **GB Studio MCP server (skills vs tools, education)** | github.com/eoinjordan/gb-studio-agent |

Images: board photo from Thundercomm's RUBIK Pi 3 product page; headshot and badges mine; screenshots and recordings taken on the Pi during the demos; blog images from my Edge Impulse write-up; diagrams drawn in draw.io from scripts in the repo and the animations in HTML, both built with AI assistance (Claude); the bottling-line belt from the Edge Impulse Unity example.

# Backup: the toolbox, learned for this talk

## mongosh, the shell

```
db.runs.findOne({ runId })
createIndex({ projectId: 1, status: 1 })
find(...).explain("executionStats")
updateOne({ _id }, { $set: { status } })
createCollection + expireAfterSeconds
runs.watch([{ $match: ... }])
aggregate([...]) over stored vectors
```

## Python

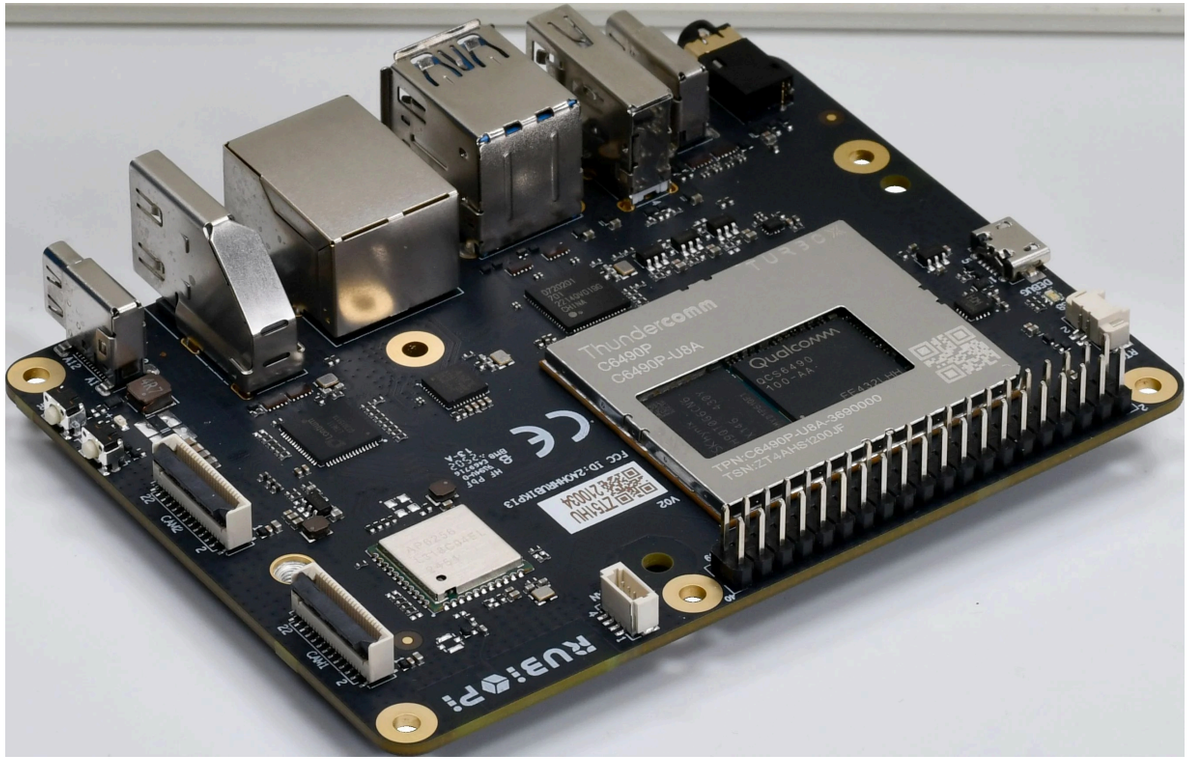
```
pymongo: MongoClient, find_one, insert_one, update_one
watch(pipeline, full_document="updateLookup",
      resume_after=token)
DuplicateKeyError as the idempotency check
langgraph-checkpoint-mongodb: MongoDBSaver(thread_id),
      MongoDBStore(namespace)
langchain_openai: ChatOpenAI(base_url=local server)
embeddings: voyage-3-lite, or bge-small on the Pi
```

## Around it

- MongoDB MCP server + Agent Skills
  - npx skills add mongodb/agent-skills
- llama.cpp server on the Pi: OpenAI-compatible,
  - json_schema responses
- MongoDBSimCity: what the server does underneath
- mongosh scripts in demo/, Python consumers beside them

*New to me three weeks before this talk: mongosh, PyMongo, the LangGraph MongoDB checkpointer. Every demo is the first thing I wrote with each of them.*

# Backup: the board on the desk




RUBIK Pi 3 by Thundercomm, Qualcomm Dragonwing QCS6490, native Ubuntu 24.04.

**Qualcomm Academy:** RUBIK Pi 3 introduction and unboxing
academy.qualcomm.com/course-catalog

## Spec sheet

**CPU:** Kryo 670, 8 cores: 4 Cortex-A78 up to 2.7 GHz, 4 Cortex-A55 up to 1.9 GHz

**AI engine:** Hexagon NPU, 12 TOPS, 6 nm

**GPU:** Adreno 643

**Memory:** 8 GB LPDDR4x (7.1 GiB usable)

**Storage:** 128 GB UFS, M.2 NVMe slot underneath

**I/O:** HDMI 4K, dual MIPI CSI camera, 3 USB-A, USB-C, Gigabit Ethernet, Wi-Fi and Bluetooth 5, 40-pin header

**OS:** Ubuntu 24.04.3 LTS, kernel 6.8 (qcom)

## MongoDB 9.0.2 Community on it, measured

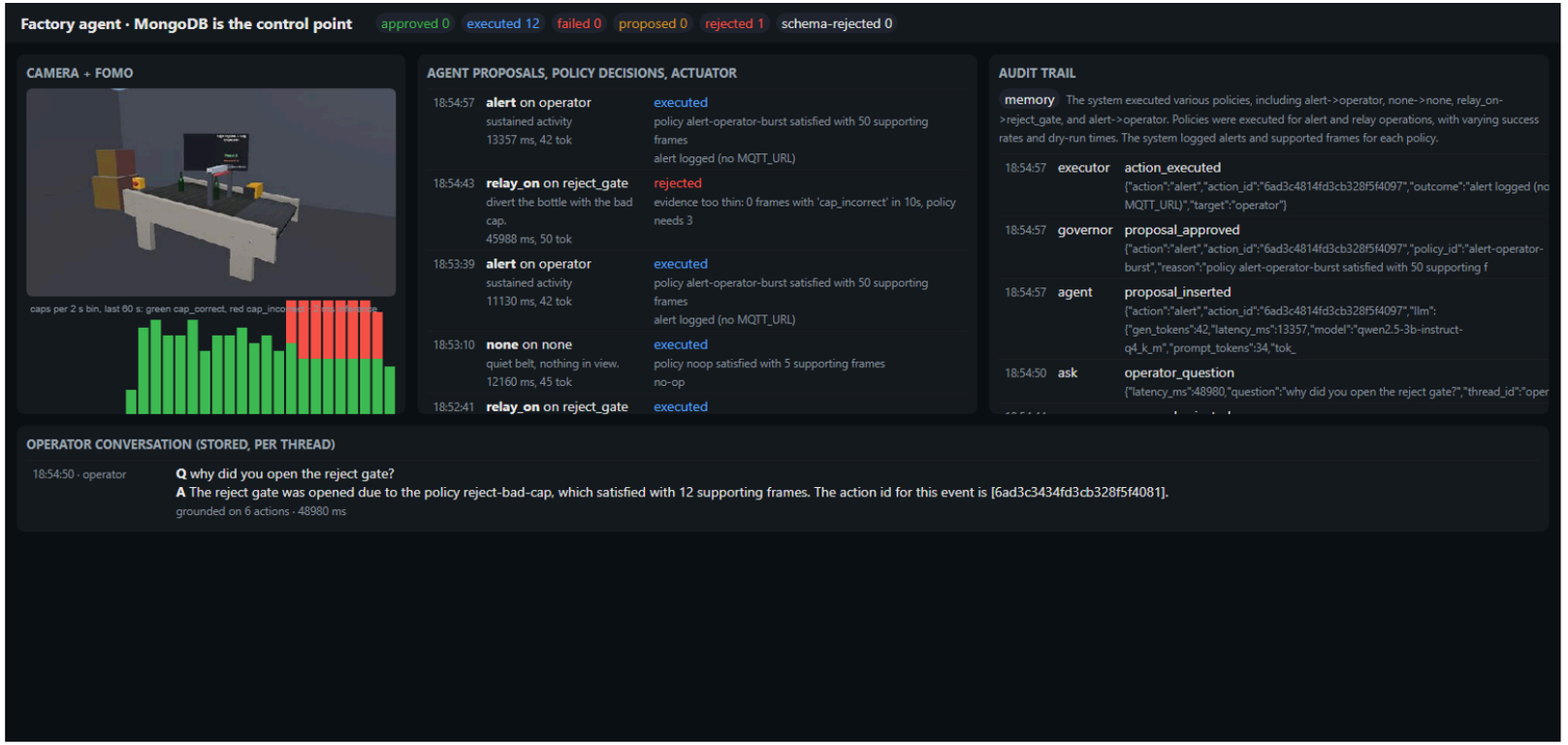
**Memory:** 270 MB resident with the demo databases open; the WiredTiger cache may grow to 3.1 GB (half of RAM minus 1 GB) and sits at about 110 MB

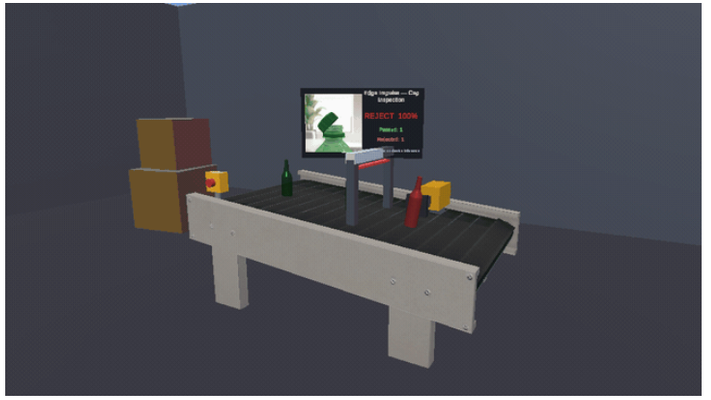
**CPU:** 5% of one core idle, 14% peak and 6% average across the full demo run (13 checks, change streams, TTL sweep)

**Disk:** 951 MB, of which 885 MB is the replica-set oplog; the demo databases are under 1 MB

**Threads:** 106, 8 connections open

# Backup: the same design on a bottling line



Camera and model write events. The agent proposes. The governor decides from the device's own evidence. The executor fires only on an approval.

Same collections, same policy gate, hardware on the end: a reject gate instead of a training job.

The factory version of this talk, running on the same RUBIK Pi: bottles with a bad cap on the Unity factory-floor belt, a 3B model proposing, MongoDB deciding. Repo on request.

# Backup: full disclosure

## What I know well

Agents that call tools, and MCP servers: the write-up this talk started from

ML on small devices: four years at Edge Impulse, now Qualcomm, on this very board

Distributed systems: eight years at IBM Galway, five as technical lead

Semantics and knowledge graphs: the PhD, and why slide 17 matters to me

## Three weeks old

mongosh and PyMongo, first used for this talk

Change streams with resume tokens, in anger rather than in the docs

Time series, TTL, schema validation and roles as a working set

The LangGraph MongoDB checkpointer and store

Two skill badges this month; Schema Design Patterns in progress

## What I have not done

Run this beyond one board: no fleet, no sharded cluster in production

Atlas Vector Search with mongot; on the Pi similarity is an aggregation

Tuned WiredTiger for real load; SimCity is a model, not a benchmark

Where any of this matters to your question, I will say so

**Why this exercise:** the brief asked for a teaching session on an advanced database topic, connected to MongoDB, for engineers new to it. I picked the problem I actually hit, an agent that could not say what it did five minutes ago, and learned the MongoDB side to answer it. Everything shown runs on the Pi on the desk and is in the repo; the claims are the ones I could test.